

Sistemas de Banco de Dados Vetoriais

Enzo Furegatti Spinella¹, Ísis Ardisson Logullo¹, Luiz Fernando de Freitas Fernandes¹,
Matheus Rodrigues da Silva espalaor¹

¹Instituto de Matemática, Estatística e Ciência da Computação - Universidade de São Paulo (USP)

enzo.spinella@usp.br, isis.logullo@usp.br

luiz.f.f.fernandes@usp.br, matheus.espalaor@usp.br

Abstract. *The continuous growth of unstructured data has led to a paradigm shift in modern computing, heavily accelerated by Deep Learning and Transformer-based models. These architectures map complex semantics into high-dimensional numerical representations known as embeddings. However, traditional relational and NoSQL database management systems fail to efficiently index and query billions of dense vectors due to the curse of dimensionality. This phenomenon invalidates geometric pruning in spatial trees, forcing inefficient exhaustive linear scans. To address this infrastructure bottleneck, Vector Database Management Systems have emerged. This paper provides a structured comparative analysis of vector databases, exploring similarity metrics and Approximate Nearest Neighbor (ANN) algorithms to support real-time applications such as RAG, multimodal search, and advanced recommendation systems.*

Resumo. *O crescimento contínuo de dados não estruturados gerou uma transição de paradigma na computação moderna, impulsionada por modelos de Deep Learning e Transformers. Essas arquiteturas mapeiam a semântica de dados complexos em representações numéricas contínuas chamadas embeddings. No entanto, os SGBDs tradicionais (relacionais e NoSQL) falham ao indexar e consultar bilhões de vetores densos devido à maldição da dimensionalidade. Esse fenômeno invalida a poda geométrica em árvores espaciais, forçando varreduras lineares exaustivas e ineficientes. Para mitigar esse gargalo de infraestrutura, surgiram os Sistemas de Bancos de Dados Vetoriais. Este trabalho apresenta uma análise comparativa dessas ferramentas, explorando métricas de similaridade e algoritmos de Vizinhos Mais Próximos Aproximados (ANN) para sustentar aplicações em tempo real como RAG, busca multimodal e recomendação.*

1. Introdução

A consolidação de modelos de *Deep Learning* e arquiteturas baseadas em *Transformers* transformou o panorama da computação moderna ao permitir a conversão de grandes volumes de dados não estruturados em representações numéricas densas, conhecidas como *embeddings*. Esse cenário gerou uma mudança de paradigma na infraestrutura de armazenamento, uma vez que os Sistemas de Gerenciamento de Banco de Dados (SGBD) tradicionais enfrentam limitações severas de desempenho ao lidar com buscas de alta dimensionalidade em larga escala.

Para solucionar esse gargalo tecnológico, surgiram os Sistemas de Gerenciamento de Bancos de Dados Vetoriais. Estas soluções utilizam algoritmos especializados de Busca por Vizinhos Próximos Aproximados (*Approximate Nearest Neighbor Search* – ANNS) para realizar varreduras semânticas eficientes em tempo real, viabilizando aplicações modernas como Geração Aumentada por Recuperação (RAG) e sistemas de recomendação avançados.

O objetivo deste trabalho é apresentar uma análise comparativa estruturada dessas tecnologias sob duas perspectivas. Inicialmente, realiza-se uma avaliação empírica do desempenho prático e do comportamento arquitetural da extensão *pgvector* (PostgreSQL) em contraposição ao banco nativo vetorial Qdrant. Em seguida, investiga-se o Estado da Arte de soluções corporativas de hiperescala através do mapeamento das plataformas Milvus e Pinecone.

O restante deste relatório está organizado da seguinte forma: a Seção 3 aborda a fundamentação teórica, detalhando as métricas de distância e os algoritmos de indexação; a Seção 12 descreve o ambiente e a metodologia dos testes práticos executados; a Seção 13 expõe e discute os resultados experimentais obtidos; a Seção 6 traça a análise comparativa do Estado da Arte; e, por fim, a Seção 14 apresenta as considerações finais e conclusões deste estudo.

2. Contextualização e Motivação

2.1. O Boom dos Modelos de Linguagem de Grande Porte (LLMs) e Dados Não Estruturados

A computação moderna enfrenta uma transição de paradigma na natureza dos dados gerados globalmente. Estima-se que mais de 80% das informações produzidas por corporações e usuários consistam em dados não estruturados, tais como textos em linguagem natural, imagens, áudios e fluxos de vídeo. Historicamente, a extração desses valores dependia de metadados textuais preenchidos manualmente ou técnicas rígidas de processamento de linguagem natural baseadas em palavras-chave.

A ascensão dos modelos de Deep Learning, consolidados pelas arquiteturas de **Transformers**, revolucionou esse cenário. Redes neurais modernas são capazes de mapear a semântica latente de dados complexos e traduzi-los em representações numéricas contínuas conhecidas como **Embeddings** de alta dimensionalidade. Desse modo, dois blocos de texto com construções sintáticas completamente distintas são projetados em regiões geometricamente próximas dentro de um espaço vetorial, permitindo que computadores executem a **busca semântica**.

Os **Large Language Models (LLMs)** representam o ápice dessa evolução no aprendizado de representações ponta a ponta aplicadas à linguagem natural. Estruturalmente, um LLM é um modelo autoregressivo baseado na arquitetura Transformer, composto por bilhões de parâmetros que são treinados de forma auto-supervisionada em *corpora* (dados com curadoria, limpeza, filtragem de conteúdo ofensivo e formatação para processar de maneira padronizada) textuais massivos à escala da internet [Zhao et al. 2023].

O princípio fundamental de um LLM reside na tarefa de modelagem de linguagem causal, cujo objetivo matemático é prever a probabilidade do próximo token dado

um histórico de tokens anteriores. Ao otimizar essa função de perda em larga escala através do mecanismo de Self-Attention, o modelo não apenas decora regras gramaticais, mas desenvolve propriedades emergentes — capacidades cognitivas complexas e comportamentos de raciocínio que não foram explicitamente programados, surgindo puramente devido à escala de parâmetros e volume de dados. Dessa forma, os LLMs atuam como motores de inferência capazes de compreender contextos sutis, responder a instruções em linguagem natural e mapear conceitos abstratos no espaço latente de seus embeddings.

Até meados de 2017, o processamento de dados sequenciais baseava-se em Redes Neurais Recorrentes e suas variantes. Essas redes processavam o texto de forma estritamente sequencial, token por token, o que gerava dois gargalos intransponíveis: a impossibilidade de paralelização do treinamento em larga escala e o problema do desvanecimento do gradiente, limitando a capacidade de reter dependências contextuais de longo alcance no texto.

A arquitetura **Transformer**, introduzida por Vaswani [Vaswani et al. 2017] e consolidada em pesquisas recentes, rompeu essa limitação ao eliminar completamente a recorrência e basear-se exclusivamente no Mecanismo de Self-Attention. O Self-Attention permite que cada token em uma sequência olhe para todas as outras palavras simultaneamente, calculando dinamicamente um peso de relevância mútua, independentemente da distância física entre elas no texto.

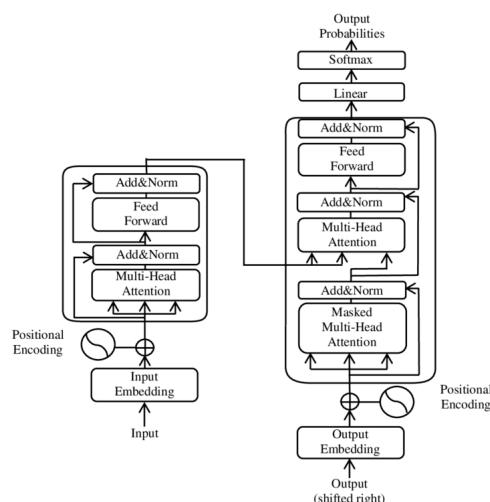


Figura 1. Estrutura Transformer

Como as arquiteturas de computadores e os modelos matemáticos operam estritamente sobre matrizes numéricas, é necessário traduzir entidades discretas do mundo real (palavras, imagens pixeladas, áudios) em entidades contínuas. Um **Embedding** é uma representação vetorial densa de uma entidade em um espaço contínuo de alta dimensionalidade, projetada de modo que a estrutura geométrica desse espaço reflita as propriedades semânticas subjacentes do domínio original.

O **Deep Learning** é um subcampo do Machine Learning fundamentado em arquiteturas de Redes Neurais Artificiais com múltiplas camadas ocultas interconectadas. Ao contrário dos algoritmos tradicionais de aprendizado supervisionado, que dependem criticamente de engenharia de atributos manual, o Deep Learning introduz o conceito de

aprendizado de representação ponta a ponta.

2.2. Limitações dos SGBDs Tradicionais

Os Sistemas de Gerenciamento de Bancos de Dados (SGBDs) tradicionais, sejam relacionais (SQL) baseados na álgebra relacional ou NoSQL (documentais, chave-valor) baseados em hashing e árvores B+, foram projetados para responder a correspondências exatas, ordenação escalar e lógica booleana.

Quando confrontados com a necessidade de indexar e consultar bilhões de vetores densos, os índices tradicionais falham [Malkov and Yashunin 2020]. A indexação em árvores multidimensionais tradicionais (como R-Trees ou K -d-Trees) sofre uma degradação exponencial de desempenho à medida que o número de dimensões aumenta, um fenômeno conhecido na literatura como a **maldição da dimensionalidade**. Como consequência, o SGBD é forçado a recorrer a uma busca de varredura sequencial, tornando o tempo de resposta impraticável para aplicações em tempo real.

O advento dos modelos de Deep Learning introduziu a necessidade de indexar e consultar bilhões de vetores densos (como os embeddings de alta dimensionalidade). Quando confrontados com essa demanda, os índices e paradigmas tradicionais colapsam por completo.

Somando-se à ineficiência algorítmica, há uma grave limitação de hardware e arquitetura de memória nos SGBDs tradicionais. Os algoritmos de cálculo de distância vetorial exigem acessos massivos e contínuos à memória RAM ou, pior, paginação de disco. Como os índices tradicionais não foram desenhados para paralelização a nível de instrução em nível de CPU/GPU (técnicas como SIMD - Single Instruction, Multiple Data), o processamento dessa varredura linear torna o tempo de resposta proibitivo para sistemas industriais que exigem subsegundos ou milissegundos para aplicações em tempo real.

2.3. Aplicações Modernas e a Infraestrutura Vetorial

Para contornar essa limitação e sustentar a nova onda de aplicações de Inteligência Artificial, surgiram os **Sistemas de Bancos de Dados Vetoriais**. Eles atuam como a infraestrutura central para o armazenamento, indexação persistente e recuperação eficiente de vetores de características. Entre as aplicações mais proeminentes, destacam-se:

- **Retrieval-Augmented Generation (RAG)** [Lewis et al. 2020]: Esta arquitetura resolve uma das principais limitações das grandes barreiras de modelos de linguagem (LLMs) - a volatilidade do conhecimento estático de seus pesos de treinamento e a propensão a alucinações. O processo opera em um fluxo bidirecional. Primeiro, no estágio de ingestão, documentos corporativos ou bases de dados textuais são fragmentados e convertidos em vetores densos para popular o banco vetorial. No momento da inferência, a consulta do usuário é submetida ao mesmo modelo de embedding, e o banco vetorial executa uma busca por similaridade de cosseno ou produto interno para extrair os fragmentos de texto semanticamente mais relevantes. Esse contexto recuperado é injetado dinamicamente no prompt enviado ao LLM. Como resultado, o modelo gera uma resposta fundamentada em evidências em tempo real, delimitada pelo contexto provido, sem a necessidade de re-treinamento ou *fine-tuning* dispendiosos.

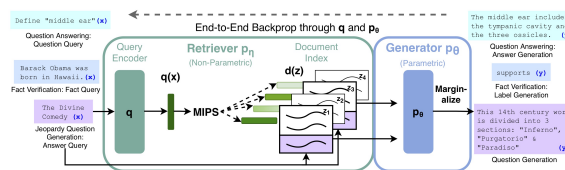


Figura 2. Fluxo RAG

- **Busca Semântica Multimodal:** Sistemas de busca tradicionais dependem estritamente de metadados textuais estruturados ou correspondências exatas de tokens. A busca multimodal rompe essa barreira ao unificar dados de diferentes naturezas — como textos, imagens, áudios e vídeos — dentro de um mesmo espaço vetorial latente compartilhado. Isso é viabilizado por codificadores bi-encoder (como a arquitetura CLIP [Radford et al. 2021]). Esses modelos são treinados para projetar conceitos equivalentes em coordenadas geométricas próximas, independentemente da mídia de origem. Consequentemente, um vetor derivado de uma descrição textual e o vetor extraído de um quadro de vídeo contendo essa exata cena terão uma distância euclidiana mínima no espaço multidimensional. O banco de dados vetorial atua como o motor que indexa esse espaço comum, permitindo consultas cruzadas instantâneas entre mídias díspares.

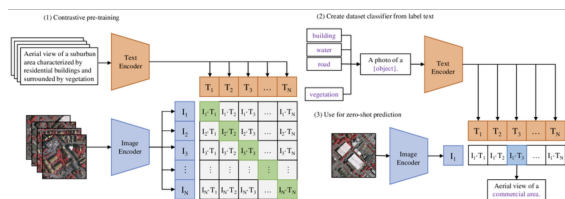


Figura 3. Arquitetura CLIP

- **Sistemas de Recomendação de Próxima Geração:** Os modelos clássicos de recomendação baseados em filtragem colaborativa ou matrizes de esparsidade sofrem com o problema de *cold start* e escalabilidade. Os sistemas modernos contornam isso mapeando de forma contínua o comportamento, preferências e histórico do usuário, bem como os atributos textuais e visuais dos itens (produtos, músicas, artigos), em vetores densos contínuos. Ao alinhar os vetores de perfil do usuário e os vetores de catálogo no mesmo espaço geométrico, a recomendação deixa de ser um cruzamento massivo de tabelas relacionais e passa a ser um problema de vizinhança topológica [Covington et al. 2016]. O banco vetorial computa e atualiza essas relações em milissegundos por proximidade geométrica. Isso garante que, mesmo diante de catálogos dinâmicos com milhões de itens, o sistema consiga entregar recomendações personalizadas com latência ultra-baixa no momento em que o usuário interage com a plataforma.

3. Fundamentos Matemáticos das Bases Vetoriais

3.1. Transformers

O modelo adota uma estrutura clássica de Codificador-Decodificador (Encoder-Decoder).

3.1.2. Self-Attention de Múltiplas Cabeças

Em vez de calcular a atenção apenas uma vez com as dimensões completas é mais vantajoso projetar linearmente as matrizes Q, K, V h vezes em subespaços de dimensões menores (d_k, d_k, d_v). Isto permite ao modelo focar simultaneamente em diferentes tipos de informação em posições diferentes.

As fórmulas são:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Onde cada cabeça (head_i) é calculada como:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$: São matrizes de projeção linear (pesos que o modelo aprende durante o treino) para a cabeça i .

$W^O \in \mathbb{R}^{h \cdot d_v \times d_{\text{model}}}$: Uma matriz de pesos finais pela qual o resultado concatenado de todas as cabeças é multiplicado para retornar à dimensão original do modelo ($d_{\text{model}} = 512$).

3.1.3. Feed-Forward Networks (FFN)

Além das camadas de atenção, cada camada do codificador e do decodificador contém uma rede totalmente conectada (Feed-Forward) que é aplicada a cada palavra de forma independente e idêntica. Ela consiste em duas transformações lineares com uma ativação ReLU no meio:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

x : O vetor da palavra na posição atual após passar pela camada de atenção.

$\max(0, \cdot)$: Esta é a função de ativação ReLU (Rectified Linear Unit), que introduz não-linearidade no modelo. Se o valor for negativo, torna-se 0; se for positivo, mantém-se igual.

W_1, b_1, W_2, b_2 : São os pesos e bias das duas camadas lineares. A dimensão interna da FFN é $d_{ff} = 2048$, enquanto a de entrada/saída é $d_{\text{model}} = 512$.

3.1.4. Codificação Posicional

Como o Transformer não utiliza recorrência (RNNs) nem convoluções (CNNs), ele não sabe a ordem das palavras numa frase. Para resolver isto, o modelo adiciona uma Codificação Posicional aos embeddings:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

pos : É a posição física da palavra na frase.

i : É o índice da dimensão dentro do vetor (varia de 0 até $d_{model}/2$). As dimensões pares ($2i$) usam a função seno, e as ímpares ($2i + 1$) usam a função cosseno.

3.1.5. Considerações do Bloco Encoder-Decoder

Cada subcamada (tanto Atenção como FFN) em cada bloco do Transformer emprega uma Conexão Residual seguida por uma Normalização de Camada (Layer Normalization):

$$\text{Saída} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Isto ajuda no fluxo de gradientes durante a backpropagation, permitindo treinar redes muito profundas sem que os gradientes desapareçam ou explodam.

3.2. Embeddings

Seja $\mathcal{D}$ um espaço amostral de dados não estruturados (e.g., um corpus de texto textualmente infinito ou um conjunto discreto de tokens). Um modelo de embedding é definido como um operador de mapeamento não linear parametrizado $\phi : \mathcal{D} \rightarrow \mathcal{H}_d$, onde $\mathcal{H}_d \subseteq \mathbb{R}^d$ denota um espaço de Hilbert real de alta dimensionalidade $d \in \mathbb{N}^+$.

Para cada elemento $x \in \mathcal{D}$, o operador ϕ injeta as propriedades semânticas latentes de x em um vetor coluna denso $\mathbf{v}$:

$$\mathbf{v} = \phi(x; \Theta) = [v_1, v_2, \dots, v_d]^T \in \mathcal{H}_d$$

onde Θ representa o tensor de parâmetros (pesos sinápticos) aprendidos pelo modelo durante a otimização de uma função de perda global $\mathcal{L}$.

Em arquiteturas do estado da arte baseadas em Transformers ($d = 768$) ou LLMs comerciais ($d = 1536$ ou $d = 3072$), a dimensão d define o grau de liberdade do espaço latente. Cada componente v_i (para $i = 1, \dots, d$) representa a projeção escalar de $\mathbf{v}$ sobre um vetor ortonormal de uma base canônica $\mathcal{B} = \{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_d\}$ que gera $\mathcal{H}_d$. Esses eixos coordenados correspondem a direções ortogonais em uma variedade latente abstrata, capturando distribuições de características sintáticas, semânticas e contextuais.

O espaço $\mathcal{H}_d$ é necessariamente dotado de uma estrutura métrica interna induzida por um produto escalar $\langle \cdot, \cdot \rangle$, tipicamente avaliada via Similaridade de Cosseno, definida formalmente como:

$$\text{Sim}(\mathbf{v}_i, \mathbf{v}_j) = \frac{\langle \mathbf{v}_i, \mathbf{v}_j \rangle}{\|\mathbf{v}_i\|_2 \|\mathbf{v}_j\|_2} = \frac{\sum_{k=1}^d v_{ik} v_{jk}}{\sqrt{\sum_{k=1}^d v_{ik}^2} \sqrt{\sum_{k=1}^d v_{jk}^2}}$$

Um Espaço de Hilbert é um espaço vetorial que possui geometria completa. Para que um espaço receba esse título, ele precisa cumprir três requisitos matemáticos rigorosos:

- Possuir um Produto Interno (Produto Escalar): Diferente de um espaço vetorial comum onde você só pode somar e multiplicar vetores por números, no espaço de Hilbert você pode multiplicar dois vetores entre si através de um operador $\langle \mathbf{u}, \mathbf{v} \rangle$.
- Induzir Métrica e Norma: A existência do produto interno permite definir o comprimento de um vetor (Norma: $\|\mathbf{u}\| = \sqrt{\langle \mathbf{u}, \mathbf{u} \rangle}$) e a distância entre dois vetores ($d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|$).
- Ser Completo (Espaço de Banach): Significa que qualquer sequência de vetores que pareça estar convergindo para algum lugar (sequências de Cauchy) realmente converge para um vetor que também está dentro desse mesmo espaço.

O espaço de embeddings possui um produto interno bem definido, podemos calcular o ângulo entre dois vetores semânticos. Isso permite usar a Similaridade de Cosseno para dizer se a palavra está geometricamente próxima.

Em alta dimensionalidade, escolher dois vetores totalmente aleatórios no espaço de Hilbert, o ângulo entre eles será quase sempre de 90° (ortogonais). Geometricamente, os vetores tendem a se isolar nos cantos e superfícies do espaço. Para que dois vetores em $d = 1536$ tenham uma similaridade de cosseno alta (próxima de 1), a rede neural precisa alinhar de forma extremamente precisa centenas de coordenadas textuais, garantindo que apenas conceitos verdadeiramente correlacionados compartilhem a mesma vizinhança geométrica.

3.3. Métricas de Distância e Similaridade Vetorial

A busca semântica em bancos de dados vetoriais consiste em encontrar os vetores logicamente mais próximos de um vetor de query $\mathbf{q} \in \mathbb{R}^d$. Para mensurar essa proximidade, utilizam-se funções que mapeiam o par de vetores para um escalar real. Sejam $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$:

3.3.1. Distância Euclidiana (L_2)

Mede a distância geométrica em linha reta entre os pontos finais de dois vetores no espaço euclidiano. É formalmente definida pela norma L_2 do vetor diferença:

$$D_{Euclidiana}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^d (u_i - v_i)^2}$$

Satisfaz todas as condições de uma métrica rígida (não-negatividade, identidade dos indiscerníveis, simetria e a desigualdade triangular: $\|\mathbf{u} - \mathbf{w}\|_2 \leq \|\mathbf{u} - \mathbf{v}\|_2 + \|\mathbf{v} - \mathbf{w}\|_2$).

3.3.2. Similaridade de Cosseno

Mede o cosseno do ângulo formado entre os dois vetores, focando estritamente na sua orientação e ignorando as suas magnitudes absolutas. É expressa pela razão entre o produto interno e o produto de suas normas:

$$S_{Cosseno}(\mathbf{u}, \mathbf{v}) = \cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \sqrt{\sum_{i=1}^d v_i^2}}$$

Fornecer valores estritamente limitados no intervalo $[-1, 1]$. Em muitos contextos práticos de banco de dados, utiliza-se a **Distância de Cosseno**, dada por:

$$D_{Cosseno}(\mathbf{u}, \mathbf{v}) = 1 - S_{Cosseno}(\mathbf{u}, \mathbf{v})$$

3.3.3. Produto Interno

Calcula a projeção de um vetor sobre o outro multiplicada por sua magnitude. É comumente utilizado quando os vetores gerados pelo modelo já estão previamente normalizados para possuírem norma unitária ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$):

$$IP(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^d u_i v_i$$

Se os vetores estão na esfera unitária, o Produto Interno é matematicamente equivalente à Similaridade de Cosseno, reduzindo drasticamente o custo computacional por eliminar o cálculo das raízes quadradas das normas no denominador.

3.4. Maldição da Dimensionalidade

A tentativa de estender as estruturas de particionamento espacial para o domínio multidimensional por meio de árvores como as K -d-Trees ou R-Trees falha categoricamente devido ao fenômeno matemático conhecido como a maldição da dimensionalidade [Bellman 1961].

Nas estruturas tipo K -d-Trees, o espaço é particionado de maneira cíclica ao longo das coordenadas cartesianas através de hiperplanos ortogonais. À medida que o número de dimensões (d) cresce, o número de regiões (hipercubos) necessárias para cobrir o espaço amostral cresce exponencialmente à taxa de 2^d [Weber et al. 1998]. Consequentemente, para um vetor de busca em alta dimensão (onde d frequentemente excede 512 ou 1024), quase todos os caminhos e subárvores da estrutura precisam ser inspecionados durante uma consulta de vizinhos mais próximos (k -NN).

Considere um hipercubo d -dimensional de aresta de comprimento 1. Seu volume total é dado por $V_{cubo} = 1^d = 1$. Se definirmos uma esfera inscrita nesse cubo com raio $r = 0.5$, o volume dessa hipersfera é definido por:

$$V_{esfera}(d) = \frac{\pi^{\frac{d}{2}}}{\Gamma\left(\frac{d}{2} + 1\right)} r^d$$

Onde Γ representa a função Gamma. Conforme $d \rightarrow \infty$:

$$\lim_{d \rightarrow \infty} V_{esfera}(d) = 0$$

Esse resultado demonstra analiticamente que, em altas dimensões, o volume de uma hipersfera inscrita torna-se insignificante em relação ao volume do hipercubo que a encerra. Geometricamente, isso implica que quase todo o volume de um hipercubo se concentra nos seus vértices.

Como consequência direta para bancos de dados, os pontos de dados distribuídos aleatoriamente em um espaço de alta dimensão tornam-se equidistantes uns dos outros. A distância máxima e a distância mínima entre quaisquer pares de vetores convergem para o mesmo valor:

$$\lim_{d \rightarrow \infty} \frac{D_{max} - D_{min}}{D_{min}} = 0$$

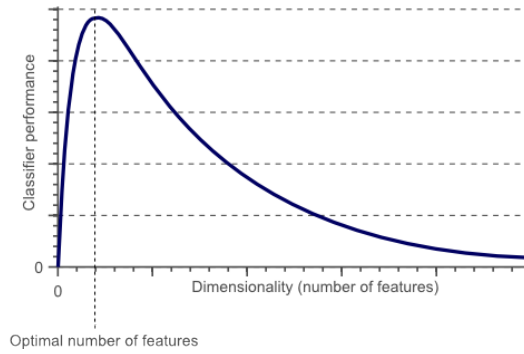


Figura 5. Maldição da Dimensionalidade

Onde D_{max} e D_{min} representam as distâncias máxima e mínima de um ponto de consulta para os pontos do banco de dados, respectivamente. Como a distância entre os vetores se torna homogeneamente uniforme, o conceito de proximidade perde o sentido estatístico. Para o algoritmo de busca na árvore, isso significa que o mecanismo de poda de ramificações se torna ineficaz: o algoritmo é incapaz de descartar subárvores com base em bounding boxes, pois a esfera de busca da consulta inevitavelmente intercepta quase todos os hipercubos do índice.

Como consequência direta dessa degradação estrutural, o comportamento assintótico de busca dessas árvores multidimensionais degenera de $O(\log N)$ para $O(N)$, onde N é o número total de vetores. O SGBD é, portanto, forçado a retroceder para uma varredura sequencial linear (linear scan), calculando exaustivamente a distância (seja Euclidiana, Cosseno ou Produto Interno) entre o vetor de consulta e todos os bilhões de vetores da base de dados. É por este exato motivo que os sistemas de bancos de dados vectoriais abrem mão da exatidão absoluta e adotam algoritmos de Vizinhos Mais Próximos Aproximados (ANN), trocando uma fração infinitesimal de precisão por ganhos de ordens de magnitude em latência e vazão.

4. Algoritmos de Vizinhos Mais Próximos Aproximados (ANN)

Algoritmos ANN podem ser pensados como buscas de vizinhos mais próximos (k-NN) que não necessariamente retornam esse vizinho mais próximo, mas sim algum ponto dentro de uma margem de erro em volta dele. Mais formalmente, seja $D : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$

uma métrica de distância e $P \subset \mathbb{R}^d$ um conjunto de n pontos. Um algoritmo para um problema ϵ -ANN recebe como entrada um ponto $q \in \mathbb{R}^d$ e devolve um ponto $p \in P$ tal que, para todo $p' \in P$, $D(p, q) \leq (1 + \epsilon)D(p', q)$. Alguns algoritmos desse tipo encontraram sucesso em mitigar a maldição da dimensionalidade, os principais sendo o *Hierarchical Navigable Small World* e o *Inverted File Index* com quantização de produto.

4.1. Hierarchical Navigable Small World (HNSW)

o HNSW é um aperfeiçoamento de outro algoritmo ANN chamado *Navigable Small World* (NSW), que por sua vez usa o conceito de grafos navegáveis para tentar resolver o problema eficientemente.

4.1.1. Conceitos

Um *grafo de proximidade* G é um grafo direcionado onde seus vértices representam um subconjunto de pontos de um espaço métrico e as arestas são tais que k-NN pode ser resolvido de forma gulosa. Isso significa que, dado um ponto q desse espaço, o seguinte procedimento sempre termina e devolve o vértice de G mais próximo de q : começando em um vértice arbitrário p , itera-se por todos os seus vizinhos e, se nenhum deles tem distância estritamente menor que a distância entre p e q , p é retornado, caso contrário o vizinho com a menor distância é selecionado e o processo se repete a partir dele.

Um *grafo navegável* é um grafo de proximidade no qual o algoritmo guloso termina em tempo logarítmico ou polilogarítmico em relação ao tamanho do grafo.

4.1.2. Funcionamento

O NSW organiza os índices dos vetores gerando uma aproximação de um grafo navegável. Primeiramente, é definido um limite M do número de vizinhos que um vértice pode ter, e depois cada vértice é inserido em ordem aleatória e ligado aos M vértices mais próximos que já foram inseridos. Infelizmente, o algoritmo guloso apresenta performance polilogarítmica, limitando sua utilidade.

Visando melhorar essa performance, o HNSW junta o NSW com a ideia de *skip lists*: é implementado uma aproximação de um grafo navegável multi-camadas no qual a camada base tem todos os vértices e cada vértice tem uma chance de aparecer na próxima camada que cai exponencialmente [Malkov and Yashunin 2020].

A inserção possui 3 parâmetros principais: o ponto q que será inserido, o limite M de vizinhos por vértice em uma camada, e o limite *efConstruction* de potenciais vizinhos de q . Na inserção de um novo vértice q , primeiro é sorteado l , que é a camada mais alta na qual q aparecerá. Em seguida, o algoritmo guloso é usado na camada mais superior para achar o vértice dela mais próximo de q , e é usado novamente a partir desse vértice achado no passo anterior, porém uma camada abaixo. Isso se repete até a camada l ser alcançada, a partir da qual o procedimento do NSW é feito até a camada base, porém considerando apenas *efConstruction* vértices por camada [Malkov and Yashunin 2020].

A busca pelo vértice mais próximo de um ponto q é similar à inserção: o algoritmo guloso começa de um vértice arbitrário da camada mais elevada, acha o vértice p dessa

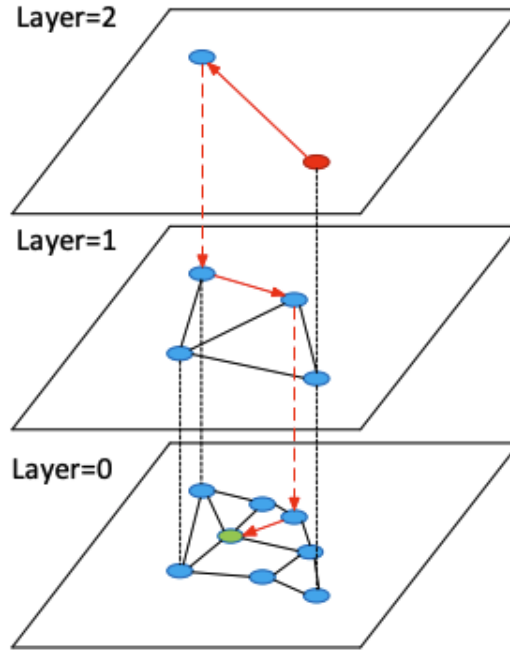


Figura 6. Exemplo de execução da busca em um HNSW. O algoritmo guloso começa na camada mais alta, acha o ponto aproximadamente ótimo dessa camada e repete o processo na camada de baixo, até chegar na última camada que tem todos os vértices.

camada mais próximo de q , e reinicia o processo a partir de p mas uma camada abaixo, até chegar na camada base.

É importante notar que, sendo $p = \exp(-m_L)$, para algum m_L , a probabilidade de um vértice aparecer na camada acima da atual, a busca nessa camada sempre acaba antes de chegar em alguns desses vértices. Isso vem das propriedades de grafos de proximidade, já que cada passo do algoritmo guloso necessariamente diminui a distância entre o ponto atual e o desejado, mas se durante a execução do algoritmo um vértice que está na camada superior é visitado, isso significa que esse vértice tem uma distância menor do que o vértice usado para descer dessa camada superior, mas esse era para ser o ótimo da camada, gerando uma contradição. Assim, a probabilidade do algoritmo guloso descer de camada depois de k passos é $p^k = \exp(-km_L)$, portanto o número de passos esperado por camada é limitado pela soma da progressão geométrica em k , que independe do tamanho do grafo ou do número de dimensões dos vetores [Malkov and Yashunin 2020]. Visto que o número de vizinhos checados por passo também é limitado por uma constante (M), o tempo de execução por camada é assintoticamente constante e, como o número de camadas esperado é $O(\log(N))$ (já que depende exponencialmente de p), o tempo de execução esperado da busca é $O(\log(N))$, mostrando que realmente em relação ao tempo polilogaritmico do NSW.

4.1.3. Trade-Offs

Os dois parâmetros que mais afetam a performance do HNSW são M e $efConstruction$. Como $efConstruction$ limita o número de vértices visitados em uma camada durante

uma inserção, ele tem grande impacto durante a criação dos índices. Por um lado, um maior valor de *efConstruction* implicaria uma melhor seleção de vizinhos para cada vértice, ou seja, um recall maior, mas também significa maiores tempos de criação dos índices. Por sua vez, um maior *M* é melhor para recalls maiores, porém faz com que haja mais vizinhos por vértice, resultando em um grande impacto no uso de memória.

4.2. *Inverted File Index* com quantização de produto (IVFPQ)

O IVFPQ (também chamado de IVFACD) consiste na junção do algoritmo *Inverted File Index* (IVF) com o algoritmo de quantização de produto (PQ). Esta técnica híbrida é largamente adotada por motores modernos de busca vetorial para otimizar os requisitos de armazenamento em cenários corporativos distribuídos de larga escala [Pan et al. 2023].

4.2.1. IFV

O IFV se utiliza de métodos como *k*-means para particionar os dados em clusters. Cada cluster recebe um índice e a cada índice é associada uma lista invertida, que contém os pontos que se encontram no cluster em questão. Dessa forma, ao realizar uma busca, em vez de iterar por todos os vértices, itera-se primeiramente pelos *centroides* de cada cluster, centroide sendo a média dos pontos do cluster, e depois a busca exaustiva é realizada apenas no cluster com centroide mais próximo do ponto desejado.

4.2.2. PQ

O PQ consiste em dividir cada vetor em subvetores de mesmo tamanho e quantizar cada um desses subvetores, resultando em uma versão destrutivamente comprimida do vetor original.

Mais formalmente, sendo $I = \{0, \dots, k - 1\}$ um conjunto finito de índices e $C = \{c_i, i \in I\} \subset \mathbb{R}^d$, uma função $q : \mathbb{R}^d \rightarrow \mathbb{R}^d$ é *quantizadora* se $q(x) \in C$ para algum C como definido anteriormente. Dado um valor m que divide d , o algoritmo PQ divide um vetor $x \in \mathbb{R}^d$ em m subvetores de mesmo tamanho $u_i(x) \in \mathbb{R}^{d/m}$ e, em cada um deles, aplica uma função quantizadora $q_i : \mathbb{R}^{d/m} \rightarrow \mathbb{R}^{d/m}$, $i \in \{1, \dots, m\}$. O conjunto C_i de cada q_i é determinado por treinamento, ou seja, pode-se associar um índice a cada $c_{ij} \in C_i$ e, como $q_i(u_i(x)) \in C_i$, em vez de armazenar x como uma lista de $q_i(u_i(x))$, ele pode ser armazenado como uma lista de índices, reduzindo drasticamente o espaço em memória ocupado.

Além de poupar memória, essa quantização também acelera o cálculo de distâncias entre dois pontos por meio de *computação de distância assimétrica* (ADC). Seja $x \in \mathbb{R}^d$ e y um ponto da database, ou seja, y está armazenado como $q(y)$. Uma estimativa d' de distância entre x e y é dada por:

$$d'(x, y) = d(x, q(y)) = \sqrt{\sum_j d(u_j(x), q(u_j(y)))^2}$$

Como $q(u_j(y))$ já é conhecido por já ser quantizado, ou seja, é algum c_{ij} , pode-se calcular todo $d(u_j(x), q(u_j(y)))^2$ previamente, fazendo com que apenas a soma tenha um

impacto real na performance (a raiz quadrada não precisa ser calculada na prática visto que é uma função monotônica crescente, implicando que não afeta qual ponto é mais próximo de x).

4.2.3. IVFPQ

Quando comparando o IVF e o PQ, percebe-se que seus pontos fracos são opostos: enquanto o IVF realiza uma varredura por força bruta em uma pequena parcela dos vetores, o PQ faz uma varredura mais otimizada mas exaustiva. Assim, a ideia do IVFPQ é usar os dois para mitigar essas fraquezas.

O funcionamento do IVFPQ é relativamente simples: são definidas duas funções quantificadoras q_c e q_p tais que q_c associa um ponto ao centroide de seu cluster (como no IVF) e q_p é como a função de quantização do PQ. Dessa forma, ao fazer uma busca, utiliza-se q_c para restringir o espaço de possibilidades a apenas um cluster e depois utiliza-se q_p e ADC para realizar a varredura do cluster de forma eficiente.

4.2.4. Trade-Offs

Os principais trade-offs do IVFPQ estão relacionados às funções q_c e q_p . Caso haja uma quantidade maior de clusters na imagem de q_c , a busca será mais detalhada, levando a um maior recall, porém isso implica em um maior tempo para a execução da busca. No caso de q_p , uma maior compressão implica menor uso de memória principal, porém, como essa compressão é destrutiva, isso levaria a mais perda de informações, impactando o recall.

Além disso, existe a questão dos centroides de q_p serem treinados, o que implica não só em longos tempos de construção do índices, mas também que esse centroides podem ficar desatualizados se muitos vetores novos forem inseridos, fazendo com que seja necessário fazer um retreinamento para preservar o recall.

5. Estado da Arte Teórico

Mesmo que métodos como Kd-trees ainda sejam usados para dados de baixa dimensionalidade, algoritmos baseados em índices invertidos e grafos navegáveis são amplamente utilizados em contextos de alta dimensionalidade, existindo implementações tanto do HNSW quanto do IVFPQ em grandes bibliotecas especializadas, como a FAISS do Facebook.

6. Estado da Arte

De acordo com [Pan et al. 2023], atualmente, existem mais de 20 database vector systems no mercado. Vamos focar nos mais modernos e usados atualmente no mercado e no ambiente científico.

7. Pinecone

A evolução recente dos *Vector Database Management Systems* é marcada pela transição de índices mantidos estritamente em memória para arquiteturas nativas em nuvem e sem servidor. Neste cenário, o **Pinecone** destaca-se, projetado especificamente para

suprir as demandas de latência e consistência de aplicações baseadas em LLMs e RAG. [Zhan et al. 2025].

O Pinecone, diferente dos próximos que serão apresentados, é de código fechado e é um SaaS, isto é, o usuário não se preocupa em gerenciar o sistema, a escalabilidade, nem a infraestrutura. Porém, isso vem com um alto custo monetário e o usuário fica preso ao sistema de nuvem do Pinecone.

7.1. Arquitetura Desacoplada e Armazenamento

O sistema de armazenamento utiliza estruturas de arquivos imutáveis persistidos em serviços de armazenamento de objetos distribuídos. A execução de consultas é delegada a executores, permitindo a auto-escalabilidade horizontal e independente do throughput de leitura (QPS) e do volume total de vetores armazenados, mitigando os custos operacionais de infraestrutura em escala de bilhões de vetores.

7.2. Busca Híbrida e Filtragem de Metadados em Etapa Única

Um dos principais gargalos teóricos em *Vector Database Management Systems* é a eficiência em buscas vetoriais restritas por atributos escalares (filtros de metadados). As abordagens tradicionais dividem-se em:

1. **Pré-filtragem:** Reduz o espaço de busca aplicando o filtro escalar antes da navegação no grafo ANN, o que frequentemente fragmenta a topologia do grafo e degrada o retorno dos dados.
2. **Pós-filtragem:** Executa a busca vetorial no grafo completo e descarta os resultados que não satisfazem o filtro escalar, resultando em latência desproporcional e retrabalho computacional caso a seletividade do filtro seja alta.

O Pinecone resolve esta limitação através do mecanismo de **Filtragem em Etapa Única**, onde índices invertidos e vetores de bits de metadados são integrados diretamente ao processo de travessia do algoritmo de Vizinhos Mais Próximos Aproximados (ANN). Esse fluxo unificado garante latência determinística $O(\log N)$ mesmo em cenários com filtros altamente seletivos e combinados com representações esparsas e densas simultaneamente.

8. Milvus

Diferentemente do Pinecone, o Milvus é um sistema gerenciador de código aberto. Amplamente estudado e usado no meio científico e empresarial.

8.1. Desacoplamento de Atores e Ciclo de Vida dos Dados

A inovação estrutural do Milvus reside na separação estrita entre computação, armazenamento e fluxo de mensagens de log. O sistema divide suas operações em três camadas independentes:

1. **Camada de Coordenação:** Funciona como o cérebro do cluster, gerenciando a atribuição de nós, topologia da rede, balanceamento de carga e metadados do sistema através de um motor de consenso distribuído.
2. **Camada de Execução:** Composta por nós trabalhadores especializados e *stateless*. Subdivide-se em *Query Nodes*, exclusivos para computação de consultas ANN, *Data Nodes*, responsáveis pelo consumo de logs e persistência e *Index Nodes*, dedicados à construção assíncrona de índices vetoriais pesados.

3. **Camada de Mensageria e Armazenamento:** Utiliza corretores de mensagens, como Apache Pulsar ou Kafka, para garantir a persistência de logs de escrita (*Write-Ahead Logging* - WAL) antes que os dados sejam armazenados em sistemas como Amazon S3.

8.1.1. Consistência Dinâmica e Segmentação de Dados Semânticos

Diferente dos bancos relacionais ACID clássicos, o Milvus permite ao usuário parametrizar o nível de consistência em tempo de execução, variando desde a consistência forte até a consistência eventual [Wang et al. 2021]. Para processar o fluxo contínuo de dados, o sistema introduz os conceitos de dados *Bounded*, segmentos fechados e indexados e *Unbounded*, dados recém-inseridos que residem em buffers de memória.

Durante uma consulta, o Milvus executa uma busca paralela híbrida. Temos o conjunto de vetores já consolidados em índices estruturados e os vetores em estado de *stream* ainda não indexados. A operação de busca de vizinhos mais próximos é formalizada como a união dos resultados computados sobre ambas as partições de memória.

Onde o KNN executa uma busca exaustiva de força bruta via força computacional paralela, usando núcleos CUDA em GPUs, se disponíveis, garantindo que dados recém-inseridos fiquem disponíveis para consulta em milissegundos sem interromper a estabilidade dos índices globais de larga escala [Wang et al. 2021].

9. pgvector

Diferentemente do Milvus e do Pinecone, o pgvector não foi construído do zero como um sistema puramente vetorial. Mas sim, foi construído como uma extensão do Postgres. Dessa forma, transformou o PostgreSQL em um banco de dados que suporta vetorial e relacional.

9.1. Mecanismos de Funcionamento e Indexação

A extensão introduziu o tipo nativo `vector` e operadores geométricos fundamentais para calcular a distância entre um vetor de consulta $\vec{q}$ e um vetor armazenado $\vec{x}$. É utilizado cálculo de distância simples já mencionados anteriormente, como: distância euclidiana e a similaridade de cossenos.

9.2. Algoritmos de Indexação Suportados

Para mitigar a complexidade exaustiva $\mathcal{O}(N)$, o pgvector implementa dois algoritmos principais de Busca por Vizinhos Próximos Aproximados (*Approximate Nearest Neighbor Search* - ANNS):

1. **IVFFlat:** Executa a clusterização do espaço vetorial através de k -means criando listas invertidas. Possui baixo consumo de memória e rápida construção, mas menor precisão se o número de partições examinadas for pequeno.
2. **HNSW:** Constrói um grafo hierárquico estruturado de múltiplos níveis. Possui uma ótima precisão e um ótimo tempo, embora demande maior processamento e memória RAM para a construção e manutenção do índice.

9.3. Vantagens:

- **Consistência ACID Completa:** O vetor e os dados transacionais residem no mesmo bloco de dados, garantindo atomicidade absoluta via logs de escrita antecipada (WAL).
- **Filtragem Híbrida Eficiente:** O otimizador de consultas do PostgreSQL avalia estatísticas estruturadas e índices vetoriais no mesmo plano de execução, permitindo instruções SQL complexas

9.4. Limitações Estruturais do pgvector

Apesar de sua enorme conveniência, a arquitetura baseada em extensão impõe gargalos físicos rígidos que impedem o `pgvector` de substituir sistemas massivamente distribuídos (como o Milvus) em cenários de hiperescala.

9.4.1. Gargalos e Restrições Técnicas

1. Limite de Dimensões: o que restringe o número de dimensões do embedding para 2.000, impedindo o uso de modelos científicos pesados de última geração sem a aplicação de técnicas de compressão.
2. Dependência de CPU: Depende inteiramente de instruções SIMD de CPU. Sistemas como o Milvus utilizam paralelismo massivo em GPU (Nvidia CUDA), operando ordens de magnitude acima do rendimento suportado pelo PostgreSQL em grandes volumes.
3. Acoplamento de recursos: Como o PostgreSQL opera em uma arquitetura baseada em processos estruturados para CPUs, cálculos vetoriais intensivos competem diretamente por CPU e memória RAM com transações críticas de leitura/escrita. Um pico de buscas semânticas pode exaurir os recursos e degradar a aplicação principal.

10. Qdrant

O **Qdrant** é um banco de dados de vetores projetado para buscas de vizinhos mais próximos estruturadas (*Approximate Nearest Neighbor - ANN*) com suporte nativo a filtragem de metadados. Desenvolvido em Rust, seu diferencial reside na eficiência de memória e na execução de filtros em estágio único.

10.1. Estrutura de Dados Básica

No Qdrant, a unidade fundamental de armazenamento é um ponto. Matematicamente, um ponto P é definido como uma tupla:

$$P = (id, \vec{v}, \text{payload}) \quad (1)$$

Onde:

- $id \in \mathbb{N}$ ou UUID é o identificador único do ponto.
- $\vec{v} \in \mathbb{R}^d$ é o vetor de alta dimensão (d). O Qdrant também suporta múltiplos vetores nomeados por ponto: $\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n$.
- `payload` é um objeto JSON que armazena os metadados associados (ex: marcas, preços, strings).

10.2. Métricas de Distância

Para determinar a similaridade entre o vetor de busca $\vec{q}$ e um vetor armazenado $\vec{v}$, o Qdrant calcula a distância matemática baseada na métrica escolhida na criação da coleção: como similaridade de cossenos, distância euclidiana, entre outros

10.3. O Índice de Busca: HNSW

Para evitar a busca linear $O(N)$, o Qdrant utiliza uma variação do algoritmo **HNSW** (*Hierarchical Navigable Small World*).

O HNSW constrói uma estrutura de grafos em múltiplas camadas .

1. A **Camada Superior** (L_m) possui poucos nós e conexões longas para buscas rápidas e grosseiras.
2. À medida que se desce nas camadas ($L_{m-1} \dots L_0$), o grafo se torna mais denso, permitindo o refinamento da busca até encontrar os vizinhos mais próximos na L_0), que contém todos os vetores.

10.4. Filtragem em Estágio Único

O maior diferencial do Qdrant é como ele une a busca vetorial com os filtros de *payload*. Em vez de filtrar antes ou depois da busca vetorial, o Qdrant faz a **filtragem dinâmica durante a travessia do grafo HNSW**.

Uma condição de filtro definida pelo usuário, por exemplo: `preco < 50`). Durante a busca no grafo, ao avaliar se deve avançar para um nó vizinho V_j , o algoritmo valida: se o payload do V_j satisfaz a condição do usuário

Se o nó não satisfizer a condição, a aresta é ignorada no cálculo de proximidade vetorial, garantindo que o resultado final seja matematicamente preciso e respeite estritamente as regras de negócio sem perder performance.

10.5. Otimização de Memória: Quantização

Para mitigar o alto consumo de RAM, o Qdrant implementa a Quantização Escalar. Ela converte os valores de ponto flutuante de 32 bits para inteiros de 8 bits, técnica similar a empregada em modelos de Deep Learning.

Isso reduz o tamanho do índice em até $4\times$, permitindo que bilhões de dimensões sejam processadas com eficiência de cache de CPU e consumo de memória drasticamente menor.

11. Análise Comparativa

A tabela apresenta uma consolidação visual e técnica dos principais sistemas gerenciadores de bancos de dados vetoriais analisados neste relatório.

Tabela 1. Comparativo: Linguagem Base e Algoritmos de Indexação

Sistema	Linguagem Base	Algoritmos de Indexação Suportados
Pinecone	Go / C++	Proprietário (Estrutura de Grafo / ANN).
Milvus	Go / C++ / Python	HNSW, IVF, ScaNN, ANNOY.
pgvector	C (Extensão Postgres)	HNSW, IVFFlat.
Qdrant	Rust	HNSW customizado.

Tabela 2. Comparativo: Suporte a Busca Híbrida e Tipo de Licença

Sistema	Busca Híbrida (Vetor + Metadados)	Tipo de Licença
Pinecone	Filtragem em Etapa Única nativa com vetores de bits.	Proprietária (SaaS)
Milvus	Filtragem dinâmica paralela sobre dados indexados e em <i>stream</i> .	Open Source
pgvector	Filtros relacionais e vetoriais integrados no otimizador SQL.	Open Source
Qdrant	Filtragem dinâmica em estágio único durante a travessia do grafo.	Open Source

Tabela 3. Comparativo: Modelo de Escalabilidade

Sistema	Modelo de Escalabilidade
Pinecone	Nuvem nativa, totalmente desacoplada e sem servidor (<i>Serverless</i>), focada em auto-escalabilidade horizontal.
Milvus	Altamente distribuído com separação estrita de três camadas independentes (Coordenação, Execução e Mensageria).
pgvector	Essencialmente vertical, operando acoplado diretamente aos recursos físicos e processos de CPU/RAM da instância PostgreSQL.
Qdrant	Escalabilidade horizontal nativa através de um sistema robusto e eficiente de <i>sharding</i> construído em Rust.

Tabela 4. Comparativo: Vantagens e Desvantagens dos Sistemas

Sistema	Vantagens	Desvantagens
Pinecone	Totalmente gerenciado (SaaS); infraestrutura e escalabilidade transparentes; excelente para RAG com filtragem em etapa única nativa.	Código fechado (proprietário); alto custo financeiro em larga escala; dependência total do ecossistema de nuvem do provedor (<i>vendor lock-in</i>).
Milvus	Código aberto; altamente distribuído e robusto; permite consistência dinâmica e paralelismo massivo utilizando hardware focado (GPUs/CUDA).	Arquitetura complexa com muitas partes móveis (três camadas distintas); curva de aprendizado e manutenção de infraestrutura elevadas.
pgvector	Consistência ACID completa; integração nativa com dados relacionais existentes no PostgreSQL; sem necessidade de gerenciar um novo banco de dados.	Limite físico de 2.000 dimensões; dependência estrita de CPU (sem aceleração por GPU); cálculos vetoriais intensivos competem por recursos com transações SQL críticas.
Qdrant	Desenvolvido em Rust (alta eficiência de memória); filtragem dinâmica em estágio único durante a travessia do grafo HNSW; quantização escalar nativa.	Comunidade e ecossistema de ferramentas integradas ligeiramente menores quando comparados ao Milvus ou soluções de mercado mais antigas.

12. Metodologia

Para avaliar as soluções de indexação vetorial discutidas teoricamente, foi conduzido um experimento prático e controlado comparando o desempenho da extensão *pgvector* (PostgreSQL) [pgvector Contributors 2024] com o banco de dados nativo vetorial Qdrant [Qdrant Team 2024]. A metodologia adotada consistiu na execução de testes de estresse automatizados, visando mensurar três eixos principais: tempo de indexação, latência de consulta e precisão da busca aproximada.

12.1. Ambiente de Execução

A fim de garantir a isonomia e a reprodutibilidade do experimento, ambas as ferramentas foram executadas sob restrições idênticas em um ambiente isolado utilizando contêineres Docker. Os testes foram realizados em uma máquina equipada com o sistema operacional Pop!_OS, processador Intel Core i7-9750H e 16GB de memória RAM.

Para a comunicação com os bancos de dados, desenvolveu-se um *script* em Python. A conexão com o PostgreSQL foi estabelecida via *socket* persistente utilizando a biblioteca `psycopg2`, enquanto o Qdrant foi acessado por meio de seu cliente oficial, que opera sobre o protocolo HTTP/REST [Qdrant Team 2024].

Para garantir a reprodutibilidade integral deste estudo empírico, o código-fonte em Python desenvolvido para a automação e coleta de métricas do experimento, bem

como os arquivos de configuração de infraestrutura dos contêineres (*docker-compose*), foram disponibilizados publicamente. O repositório contém as instruções detalhadas de configuração do ambiente e execução do referencial de testes

O repositório completo pode ser acessado em: <https://github.com/SQL-2026-Trabalho/ep2>

12.2. Conjunto de Dados e Geração de Embeddings

O *dataset* utilizado para popular as bases de dados foi um subconjunto de 5.000 amostras textuais extraídas do repositório público `dair-ai/emotion` [dair-ai 2021]. A conversão do texto bruto para representações vetoriais (*embeddings*) foi realizada *offline* utilizando a biblioteca `sentence-transformers`, por meio do modelo pré-treinado `all-MiniLM-L6-v2` [Sentence-Transformers Contributors 2021].

Este modelo foi selecionado por seu excelente balanço entre velocidade e qualidade semântica, gerando vetores densos de 384 dimensões. Do conjunto total, 4.900 documentos foram destinados à inserção nos bancos de dados, enquanto os 100 vetores restantes foram isolados para atuar exclusivamente como parâmetros de consulta (*queries*).

12.3. Desenho do Experimento e Métricas

Para mitigar flutuações de desempenho causadas pelo escalonador do sistema operacional ou por *cache misses*, o experimento foi desenhado para executar 30 iterações independentes ($n = 30$). Em cada rodada, os índices foram integralmente recriados. As seguintes métricas foram coletadas:

- **Tempo de Inserção e Indexação:** Tempo total (em segundos) necessário para gravar os 4.900 vetores e construir os índices de busca aproximada (HNSW em ambas as ferramentas [Malkov and Yashunin 2020]).
- **Latência de Consulta:** Tempo médio (em milissegundos) para retornar os 10 vizinhos mais próximos ($k = 10$) para o lote de 100 *queries*.
- **Recall@10:** Métrica de precisão calculada pela intersecção entre os resultados da busca aproximada e um "gabarito" exato. O gabarito foi gerado forçando o PostgreSQL a realizar uma varredura sequencial ($O(N)$) utilizando a métrica de similaridade de cosseno [pgvector Contributors 2024].

13. Resultados e Avaliação Experimental

Os dados brutos coletados nas 30 iterações foram consolidados, permitindo a extração da média (μ) e do desvio padrão (σ) para cada ferramenta, conferindo validade estatística à comparação.

13.1. Desempenho de Indexação

A Figura 7 ilustra o tempo médio exigido por cada sistema para realizar o *bulk insert* dos vetores e estruturar o grafo do índice HNSW [Malkov and Yashunin 2020].

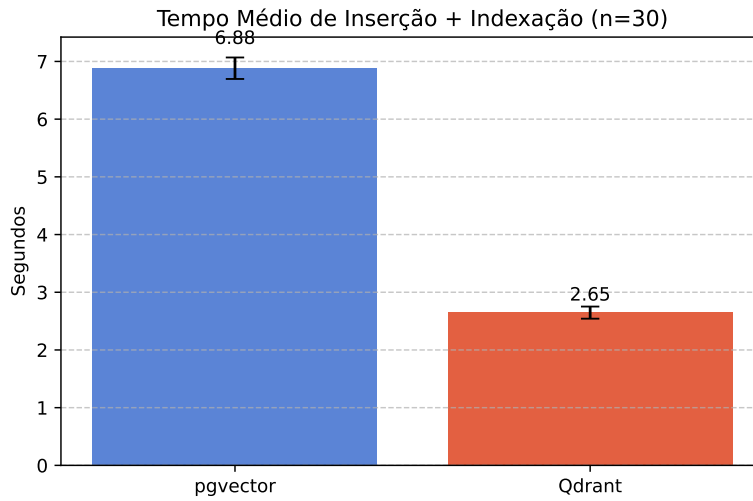


Figura 7. Comparativo do tempo médio de inserção e construção do índice HNSW ($n = 30$). As barras de erro indicam o desvio padrão observado.

O Qdrant apresentou uma vantagem expressiva nesta etapa, concluindo o processo em uma média de 2,65 segundos, contra os 6,88 segundos demandados pelo *pgvector* [Qdrant Team 2024, *pgvector* Contributors 2024]. Este resultado evidencia a otimização arquitetural do Qdrant para o processamento nativo em lotes (*batches*) [Qdrant Team 2024]. A extensão do PostgreSQL, por herdar o motor relacional padrão focado em transações ACID individuais, impõe um *overhead* maior durante operações massivas de escrita vetorial [pgvector Contributors 2024].

13.2. Latência e Precisão de Consulta

No que tange ao tempo de resposta para recuperação de informação, o cenário se inverteu, conforme demonstrado na Figura 8.

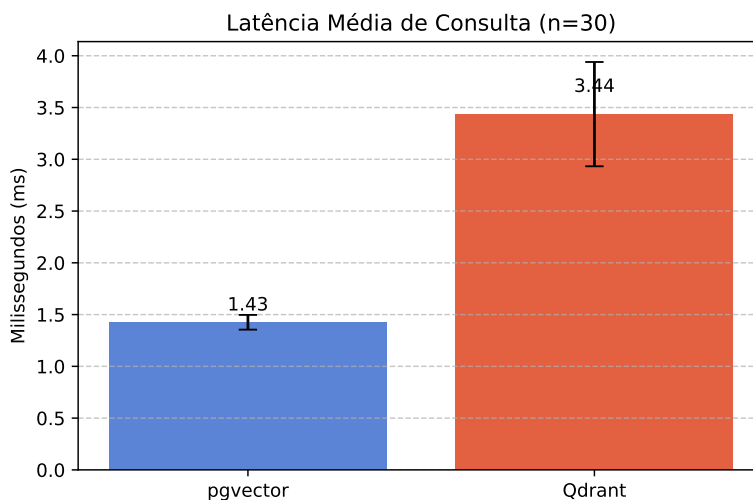


Figura 8. Comparativo da latência média por consulta ($k = 10$). As barras de erro evidenciam a maior variância na comunicação com o Qdrant.

O *pgvector* obteve uma latência substancialmente menor, registrando uma média de 1,43 ms por consulta, em contraste com os 3,44 ms observados no Qdrant [pgvector Contributors 2024, Qdrant Team 2024]. Destaca-se na Figura 8 a considerável margem de erro (σ) atrelada ao Qdrant. Esta variância superior e a maior latência média são explicadas, no contexto deste *dataset* de menor escala (4.900 registros), pelo custo inerente de rede. Enquanto o acesso ao Qdrant exige o estabelecimento e a resolução de requisições HTTP [Qdrant Team 2024], o PostgreSQL beneficia-se de conexões binárias persistentes e mais próximas do *kernel*. Em volumes de dados que não excedem a memória principal, o *overhead* de comunicação do cliente REST ofusca a velocidade de busca interna da *engine* em Rust [Qdrant Team 2024].

Por fim, a avaliação qualitativa da recuperação demonstrou que ambas as implementações do algoritmo HNSW são altamente confiáveis [Malkov and Yashunin 2020]. Ao comparar os identificadores retornados contra o gabarito de busca exaustiva, o *pgvector* atingiu um *Recall@10* médio de 98.42%, enquanto o Qdrant registrou 100.00%. Esses índices confirmam que a aceleração obtida pela aproximação do índice não compromete de forma estatisticamente significativa a correteza semântica dos resultados.

14. Conclusão

Este estudo proporcionou uma investigação abrangente sobre os Sistemas de Gerenciamento de Bancos de Dados Vetoriais, cobrindo desde seus fundamentos matemáticos e algoritmos subjacentes até sua avaliação experimental e o mapeamento do Estado da Arte na indústria. Diante do ecossistema analisado, evidencia-se que não existe uma solução universal, mas sim um balanço de compromissos técnicos atrelados à escala, custos e infraestrutura disponível.

A avaliação empírica revelou comportamentos arquiteturais distintos entre soluções integradas e motores nativos. O *pgvector*, por se consolidar como uma extensão do PostgreSQL, demonstrou ser uma alternativa extremamente viável e eficiente para conjuntos de dados de pequena e média escala. Beneficiando-se de protocolos de comunicação binária mais próximos do *kernel* e do acoplamento direto com o motor relacional, ele obteve latências de consulta inferiores às do Qdrant no cenário testado, além de garantir consistência ACID total nativa e simplificar a arquitetura de sistemas ao evitar a introdução de novos componentes na pilha tecnológica. Contudo, suas restrições de dimensionalidade e a severa concorrência por CPU evidenciam limitações para cenários de hiperescala.

Por outro lado, o Qdrant provou sua superioridade em operações de escrita em lote e eficiência de memória, impulsionado por sua implementação robusta em Rust e pelo suporte nativo à quantização escalar. Juntamente com a análise do Milvus e do Pinecone, o estudo do Estado da Arte consolidou a tendência de mercado voltada para o desacoplamento absoluto de recursos, onde camadas de computação (consultas ANN), armazenamento de objetos e mensageria distribuída escalam de forma independente e elástica em ambientes de nuvem.

Em suma, conclui-se que para aplicações corporativas que já operam sobre ecossistemas relacionais e possuem demandas moderadas de busca semântica, a extensão vetorial do PostgreSQL atende com excelência e baixo custo de manutenção. Todavia, para

sistemas que demandam bilhões de pontos vetoriais, buscas híbridas massivas restritas por metadados dinâmicos e paralelismo em GPU de altíssimo *throughput*, sistemas nativos distribuídos (como Qdrant ou Milvus) mostram-se indispensáveis.

Referências

- Bellman, R. (1961). *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton.
- Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, pages 191–198. ACM.
- dair-ai (2021). Emotion dataset.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836.
- Pan, J. J., Wang, J., and Li, G. (2023). Survey of vector database management systems. *arXiv preprint arXiv:2310.14021*.
- pgvector Contributors (2024). pgvector: Open-source vector similarity search for postgresql.
- Qdrant Team (2024). Qdrant: Vector search engine.
- Radford, A., Kim, J. W., Ramesh, C. H. A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR.
- Sentence-Transformers Contributors (2021). Pretrained models: all-minilm-l6-v2.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.
- Wang, J., Yi, X., Guo, R., Eng, A., Huang, S., Liu, H., Xiang, C., Zeng, X., Shun, Y., Li, J., et al. (2021). Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2614–2627.
- Weber, R., Blott, S., and Schek, H.-J. (1998). A quantitative analysis and performance study for similarity search methods in high-dimensional spaces. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*, pages 194–205.
- Zhan, G., Kang, G., et al. (2025). Bigvectorbench: Heterogeneous data embedding and compound queries are essential in evaluating vector databases. *Proceedings of the VLDB Endowment*, 18(5):1120–1133.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Zhao, Z., Liu, P., Wen, J.-R., and Nie, J.-Y. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.